

函数

01 基础内容

02 进阶内容

1.1 函数的基本使用

引入问题

- **问题**：分别计算 1 至 10、1 至 50、1 至 100 的整数之和。根据已学的循环结构知识，可以编写如下程序：

```
total = 0
for i in range(1, 11):
    total += i
print("1至10的和为: {}".format(total))

total = 0
for i in range(1, 51):
    total += i
print("1至50的和为: {}".format(total))

total = 0
for i in range(1, 101):
    total += i
print("1至100的和为: {}".format(total))
```

1.上述三段代码有哪些相同之处，又有哪些不同之处？

2.如果需要计算 20 组不同区间的整数之和，程序将变成什么样子？

3.如果需要修改求和的计算方式，需要修改几处代码？

引入问题

- **分析**：
  - 三段代码的求和过程完全相同，仅区间的终止值不同
  - 相同的部分重复写了三次，若需求增加到 20 组，同样的代码需要书写 20 次
  - 若需要修改求和方式，每一处重复的代码都必须逐一修改，容易遗漏
- **解决思路**：
  - 将相同的求和过程封装为一个整体，并为其命名，例如 sum_to
  - 将不同的部分（终止值）作为这一整体的输入
  - 需要求和时，只需写出名称并提供不同的输入：

```
sum_to(10)
sum_to(50)
sum_to(100)
```

将一段功能封装起来、通过名称重复使用的程序结构，称为**函数**

引入问题

- **分析**：
  - 三段代码的求和过程完全相同，仅区间的终止值不同
  - 相同的部分重复写了三次，若需求增加到 20 组，同样的代码需要书写 20 次
  - 若需要修改求和方式，每一处重复的代码都必须逐一修改，容易遗漏
- **解决思路**：
  - 将相同的求和过程封装为一个整体，并为其命名，例如 sum_to
  - 将不同的部分（终止值）作为这一整体的输入
  - 需要求和时，只需写出名称并提供不同的输入：

```
sum_to(10)
sum_to(50)
sum_to(100)
```

将一段功能封装起来、通过名称重复使用的程序结构，称为**函数**

函数的概念

- **定义**：
  - 函数是一段具有特定功能、可以被重复使用的代码块，通过函数名进行标识和调用
- **函数的基本要素**：
  - 以引入问题中的 sum_to 为例
    - 函数名：标识函数的名称，如 sum_to
    - 参数：调用函数时传入的数据，如终止值 10、50、100
    - 返回值：函数执行结束后传回调用处的结果，如 sum_to(10) 的结果 55

函数的概念

- **特点**：
  - 封装：将实现某一功能的代码组织为一个整体
  - 复用：一次定义，多次调用，每次调用可以传入不同的参数
  - 抽象：调用者只需了解函数的功能、参数和返回值，无需了解其内部实现

函数的定义

- **语法格式**：

```
def 函数名(参数列表):
    函数体
    return 返回值
```

- **说明**：
  - def：定义函数的关键字
  - 函数名：命名规则与变量名相同
  - 参数列表：可以包含零个、一个或多个参数，参数之间用逗号分隔；没有参数时，圆括号不可省略
  - 冒号与缩进：函数体中的语句须相对于 def 所在行缩进，缩进的语句均属于函数体
  - return 语句：用于结束函数并将结果返回给调用处；函数中没有 return 语句时，返回空值 None

函数的定义

- ⚠ 定义函数时，函数体中的语句并不执行，只有在调用函数时才会执行

```
print("程序开始")

def greet():
    print("函数体被执行")

print("函数定义完成")
greet()
greet()
print("程序结束")
```

运行结果：

```
程序开始
函数定义完成
函数体被执行
函数体被执行
程序结束
```

函数的调用

- **语法格式**：

函数名(参数列表)

- **形参与实参**：
  - 形式参数（简称形参）：定义函数时圆括号中的参数，用于接收调用时传入的数据
  - 实际参数（简称实参）：调用函数时圆括号中给出的具体数据

```
def square(x):
    return x * x


print(square(3))
print(square(5))
```

- x 为形参
- 3 和 5 为实参；两次调用传入不同的实参，得到不同的结果

函数的调用

- 示例：计算圆的周长：

函数名　形参

```
def circle_c(radius):
    c = 2 * 3.14159 * radius
    return c
```

函数体

返回值

实参

```
r = 10
print("半径为{}的圆的周长为: {:.2f}".format(r, circle_c(r)))
```

函数调用

函数的调用

- **示例：用函数改写求和程序：**

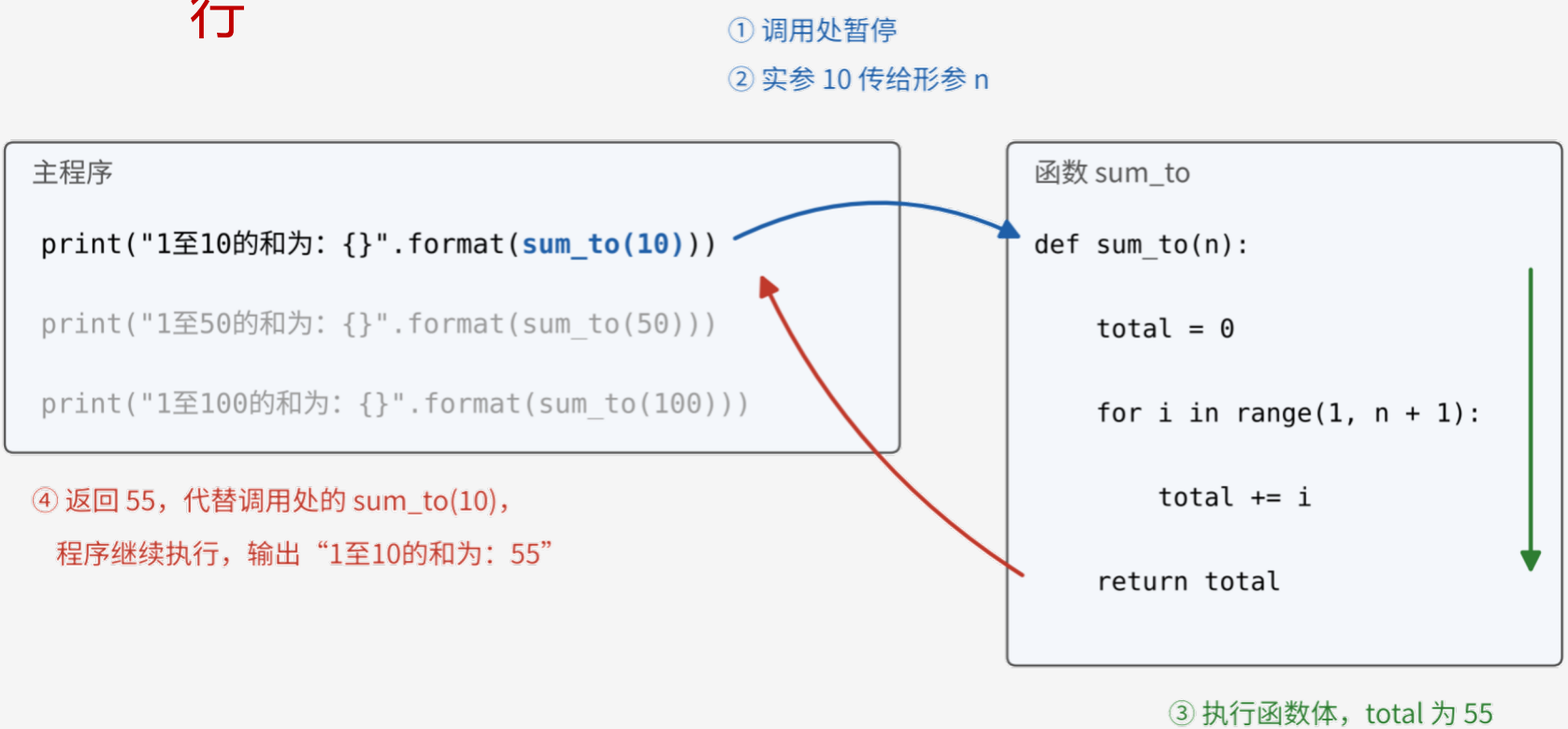
```
def sum_to(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total

print("1至10的和为: {}".format(sum_to(10)))
print("1至50的和为: {}".format(sum_to(50)))
print("1至100的和为: {}".format(sum_to(100)))
```

- 求和过程只在函数中编写一次，三次计算通过三次调用完成
- 增加一组区间的求和，只需增加一行调用语句
- 修改求和方式，只需修改函数体一处

函数的调用

- **调用步骤：**
  1. 调用程序在调用处暂停执行
  2. 将实参传递给函数的形参
  3. 执行函数体中的语句
  4. 函数执行结束，返回值传回调用处，程序从暂停处继续执行



函数中调用其他函数

- 函数体中可以调用其他函数。阅读以下三段程序，判断各自的运行结果

```
def func_a():
    print("a")

def func_b():
    func_a()
    print("b")

func_b()
```

运行结果：
a
b

```
def func_b():
    func_a()
    print("b")

func_b()

def func_a():
    print("a")
```

报错 NameError: name 'func_a' is not defined，因为调用 func_b() 时 func_a 尚未定义

```
def func_b():
    func_a()
    print("b")

def func_a():
    print("a")

func_b()
```

运行结果：
a
b

函数中调用其他函数

- 函数体中可以调用其他函数。阅读以下三段程序，判断各自的运行结果

```
def func_a():
    print("a")

def func_b():
    func_a()
    print("b")

func_b()
```

```
def func_b():
    func_a()
    print("b")

func_b()

def func_a():
    print("a")
```

```
def func_b():
    func_a()
    print("b")

def func_a():
    print("a")

func_b()
```

⚠ 函数定义的先后顺序不影响程序运行，关键在于调用一个函数时，该函数必须已经定义

函数的返回值

- **return 语句的作用：**
  - 结束函数的执行，程序回到调用处继续执行
  - 将结果传回调用处
- **示例：函数中有多个 return 语句：**

```
def my_abs(x):
    if x >= 0:
        return x
    return -x

print(my_abs(5))
print(my_abs(-3))
```

⚠ 函数中可以有多个 return 语句，执行到其中任意一个时，函数立即结束

函数的返回值

- **return 语句的作用：**
  - 结束函数的执行，程序回到调用处继续执行
  - 将结果传回调用处
- **示例：函数中没有 return 语句：**

```
def greet(name):
    print("你好，{}".format(name))

result = greet("张三")
print(result)
```

运行结果：
你好，张三 None

⚠ 函数中没有 return 语句时，返回空值 None

函数的返回值

- **函数的多个返回值：**
  - return 语句可以同时返回多个值，值之间用逗号分隔，多个返回值以元组的形式传回调用处
- **示例：同时计算圆的周长和面积：**

```
def circle(radius):
    perimeter = 2 * 3.14159 * radius
    area = 3.14159 * radius * radius
    return perimeter, area

result = circle(10)
print(result, type(result))
```

```
<class 'tuple'>
```

```
c, s = circle(10)
print("周长为{:.2f}, 面积为{:.2f}".format(c, s))
```

- 用一个变量接收：得到包含全部返回值的元组
- 用多个变量接收：返回值按顺序依次赋给各个变量，变量个数须与返回值个数相同

练习题

- 编写程序计算组合数 C_m^n，计算公式为：

$$C_m^n = \frac{m!}{n! \times (m-n)!}$$

  - 定义函数 fac(num)，返回 num 的阶乘
  - 定义函数 comb(m, n)，在函数体中调用 fac 计算组合数并返回结果
  - 调用 comb 函数，输出 C_7^3 和 C_5^2 的值

练习题

- 编写程序计算组合数 C_m^n，计算公式为：

$$C_m^n = \frac{m!}{n! \times (m-n)!}$$

```
def fac(num):
    result = 1
    for i in range(1, num + 1):
        result *= i
    return result

def comb(m, n):
    return fac(m) // (fac(n) * fac(m - n))

print(comb(7, 3))
print(comb(5, 2))
```

⚠ 使用整除运算符 //，使结果为整数；若使用 /，结果为浮点数 35.0



1.2 函数的参数

位置参数与关键字参数

- 两种传参方式：
  - 位置参数：实参按顺序依次传递给形参
  - 关键字参数：以“形参名=值”的形式传入实参；由于指定了形参名称，实参的顺序可以任意调整
- 示例：判断三条边能否构成三角形：

```
def is_triangle(a, b, c):
    return a + b > c and b + c > a and a + c > b


print(is_triangle(3, 4, 5))
print(is_triangle(1, 2, 3))
print(is_triangle(c=5, a=3, b=4))
```

前两次调用使用位置参数

第三次调用使用关键字参数，实参顺序与形参顺序不同，结果不受影响

位置参数与关键字参数

- 位置参数与关键字参数的**混合使用**：
  - 调用函数时，位置参数与关键字参数可以混合使用，但位置参数必须写在关键字参数之前
  - 同一个形参只能接收一个实参，不能既按位置传入、又按名称传入

调用语句	运行结果
`is_triangle(3, c=5, b=4)`	`True`
`is_triangle(a=3, 4, 5)`	`SyntaxError: positional argument follows keyword argument`
`is_triangle(3, a=4, c=5)`	`TypeError: is_triangle() got multiple values for argument 'a'`

- 第一条语句：3 按位置传给 a，其余两个按名称传入，调用正确
- 第二条语句：位置参数写在了关键字参数之后，违反第一条规则
- 第三条语句：3 已按位置传给 a，又以 a=4 再次传入，违反第二条规则

参数的默认值

- 概念：
  - 定义函数时，可以为形参指定默认值，形式为“形参名=默认值”
  - 调用函数时，若未给该形参传入实参，则使用默认值；若传入了实参，则使用传入的值
  - 带默认值的形参必须写在不带默认值的形参之后

```
def sum_to(n, start=1):
    total = 0
    for i in range(start, n + 1):
        total += i
    return total


print(sum_to(10))
print(sum_to(10, 5))
print(sum_to(10, start=5))
```

计算从 start 到 n 的整数之和，start 默认为 1

第一次调用未传入 start，使用默认值 1，计算 1 至 10 之和

参数的默认值

- 概念：
  - 定义函数时，可以为形参指定默认值，形式为“形参名=默认值”
  - 调用函数时，若未给该形参传入实参，则使用默认值；若传入了实参，则使用传入的值
  - 带默认值的形参必须写在不带默认值的形参之后

```
def sum_to(start=1, n):
    ...
```

3.12版本以上报错 SyntaxError: parameter without a default follows parameter with a default

可变数量参数

- 概念：
  - 定义函数时，在形参名前加星号 *，该形参可以接收任意多个实参
  - 传入的多个实参被组装为一个元组，赋给该形参
  - 带星号的形参通常写在普通形参之后
- 示例：计算学生的总分，各学生的科目数不同

```
def show_total(name, *scores):
    print(scores)
    total = 0
    for s in scores:
        total += s
    print("{}的总分为: {}".format(name, total))


show_total("张三", 85, 90, 78)
show_total("李四", 92, 88)
```

第一个实参按位置传给name，其余实参全部由scores接收

可变数量参数

- ⚠ 当同时存在默认值参数和可变数量参数时，推荐将默认值参数写在可变数量参数之后
- 示例：默认值参数写在 *scores 之后（推荐）

```
def show_total(name, *scores, unit="分"):
    print(name, scores, unit)


show_total("张三", 85, 90, 78)
show_total("李四", 4, 3, 5, unit="学分")
```

运行结果：
张三 (85, 90, 78) 分
李四 (4, 3, 5) 学分

不传 unit 时使用默认值，需要修改时写 unit="学分"

可变数量参数

- ⚠ 当同时存在默认值参数和可变数量参数时，推荐将默认值参数写在可变数量参数之后
- 示例：默认值参数写在 *scores 之前（不推荐）

```
def show_total(name, unit="分", *scores):
    print(name, scores, unit)


show_total("张三", 85, 90, 78)
show_total("李四", "学分", 4, 3, 5)
show_total("王五", 4, 3, unit="学分")
```

运行结果：

张三 (90, 78) 85
李四 (4, 3, 5) 学分
TypeError: show_total() got multiple values for argument 'unit'

可变数量参数

- ⚠ 当同时存在默认值参数和可变数量参数时，推荐将默认值参数写在可变数量参数之后
- 示例：默认值参数写在 *scores 之前（不推荐）

```
def show_total(name, unit="分", *scores):
    print(name, scores, unit)


show_total("张三", 85, 90, 78)
show_total("李四", "学分", 4, 3, 5)
show_total("王五", 4, 3, unit="学分")
```

- 第一次调用：实参按位置依次分配，85 被 unit 接收，只有 90、78 进入 scores，结果错误且不报错
- 第三次调用：4 已按位置传给 unit，又以 unit= 再次传入，报错



1.3 变量的作用域

局部变量与全局变量

- 概念：
  - 全局变量：在函数之外定义的变量，在程序执行的全过程中有效
  - 局部变量：在函数内部定义的变量，包括函数的形参；局部变量仅在函数内部有效，函数执行结束后即被释放
- 示例：在函数外访问局部变量

```
def sum_to(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total


print(sum_to(10))
print(total)
```

运行结果：

```
55
NameError: name 'total' is not defined
```

局部变量与全局变量

- 在函数内读取全局变量：
  - 函数内部可以直接读取全局变量的值，无需任何声明
  - 当函数内使用的变量在函数内部没有定义时，Python 会到函数外部查找同名的全局变量
- 示例：根据及格线判断成绩是否及格

```
pass_score = 60
```

pass_score 是全局变量，在函数 is_pass 内部可以直接读取

```
def is_pass(score):
    return score >= pass_score


print(is_pass(75))
print(is_pass(52))
```

score 是形参，属于局部变量

运行结果：

True

False

局部变量与全局变量

- 在函数内对同名变量赋值：
  - 在函数内部对变量赋值时，该变量被视为局部变量
  - 即使该变量与某个全局变量同名，赋值语句也是在函数内部新建一个局部变量，不会改变全局变量的值
- 示例：在函数内修改及格线

```
pass_score = 60

def set_pass_score(new_score):
    pass_score = new_score
    print("函数内: {}".format(pass_score))

set_pass_score(70)
print("函数外: {}".format(pass_score))
```

⚠ 函数执行结束后，局部变量 pass_score 被释放

运行结果：
函数内：70
函数外：60

局部变量与全局变量

- global 关键字：
  - 若需要在函数内部修改全局变量的值，须在使用该变量之前用 global 关键字声明
  - 声明后，函数内对该变量的赋值作用于全局变量，不再新建局部变量
- 语法格式：

```
global 变量名
```

局部变量与全局变量

- 示例：用 global 修改及格线：

```
pass_score = 60

def set_pass_score(new_score):
    global pass_score
    pass_score = new_score
    print("函数内: {}".format(pass_score))

set_pass_score(70)
print("函数外: {}".format(pass_score))
```

函数内的赋值直接修改了全局变量

运行结果：
函数内：70
函数外：70



局部变量与全局变量

- 变量作用域的规则

函数内的操作	变量的性质	对全局变量的影响
定义新变量或使用形参	局部变量，函数结束后即被释放	无
只读取变量，函数内未对其赋值	读取同名的全局变量	无
对变量赋值，未使用 `global` 声明	新建局部变量，与同名全局变量互不影响	无
先用 `global` 声明，再对变量赋值	全局变量	全局变量的值被修改



1.4 lambda 函数

lambda 函数

- 定义：
  - 是一种用 lambda 关键字定义的简单函数
  - lambda 函数本身没有函数名，因此又称为匿名函数；需要多次调用时，可以将其赋给一个变量，通过变量名调用
  - 表达式的计算结果即为函数的返回值，不需要写 return
- 语法格式：

```
lambda 参数列表: 表达式
```

• 参数可以有零个、一个或多个，多个参数之间用逗号分隔
• 冒号后只能有一个表达式

lambda 函数

- 与 def 定义的对比：

比较项	def 定义的函数	lambda 函数
函数名	定义时必须给出函数名	本身没有函数名
函数体	可以包含多条语句，如赋值、循环、分支等	只能是一个表达式
返回值	通过 `return` 语句返回	表达式的计算结果即为返回值
适用场景	功能较复杂或需要多次调用的函数	功能简单、只需一行即可表达的函数

lambda 函数

- 示例：分别用两种方式定义求和函数

```
def add(x, y):
    return x + y


f = lambda x, y: x + y


print(add(10, 12))
print(f(10, 12))
print(type(f))
```

运行结果：

```
22
22
<class 'function'>
```

lambda 函数

- 应用：指定排序规则
  - 内置函数 sorted() 对序列中的元素排序，返回排序后的新列表，原序列不变
  - sorted() 的 key 参数用于指定排序规则：key 接收一个函数，排序时将每个元素传入该函数，按函数的返回值进行排序
  - key 所需的函数通常功能简单、仅使用一次，适合用 lambda 函数实现

lambda 函数

- 应用：指定排序规则

```
students = [("张三", 85), ("李四", 92), ("王五", 78)]

def get_score(student):
    return student[1]

print(sorted(students, key=get_score))
print(sorted(students, key=lambda student: student[1]))
print(sorted(students, key=lambda student: student[1], reverse=True))
```

先用 def 定义 get_score 函数，再将函数名传给 key

直接将 lambda 函数传给 key，无需单独定义函数，效果相同

运行结果：

```
[('王五', 78), ('张三', 85), ('李四', 92)]
[('王五', 78), ('张三', 85), ('李四', 92)]
[('李四', 92), ('张三', 85), ('王五', 78)]
```

lambda 函数

- 应用：指定最大值与最小值的规则
  - 内置函数 max()、min() 同样具有 key 参数，用法与 sorted() 相同
  - max()、min() 按 key 函数的返回值比较元素，返回最大或最小的元素本身
- 示例：找出成绩最高和最低的学生

```
students = [("张三", 85), ("李四", 92), ("王五", 78)]


print(max(students, key=lambda student: student[1]))
print(min(students, key=lambda student: student[1]))
```

运行结果：
('李四', 92)
('王五', 78)

⚠ max() 与 min() 返回的是整个元组，而不仅是成绩



1.5 函数的递归

函数的递归

- 概念：
  - 函数体中可以调用其他函数，也可以调用函数自身
  - 函数在定义中调用自身的方式称为递归
- 示例：阶乘的数学定义

$$n! = \begin{cases} 1, & n = 0 \\ n \times (n-1)!, & n \geq 1 \end{cases}$$

计算 n! 时需要用到 (n-1)!，即阶乘的定义中又使用了阶乘本身，这就是递归定义

函数的递归

- 两个组成部分：
  - 基例：可以直接得出结果、无需再次递归的情形，如 n = 0 时 0! = 1
  - 递归关系：将问题转化为规模更小的同类问题，如 n! = n x (n-1)!

⚠ 每一次递归都必须向基例靠近，最终到达基例而结束；若没有基例，函数将无限地调用自身

函数的递归

- 用递归实现阶乘：
  - 基例：用 if 判断 n == 0，直接返回 1
  - 递归关系：否则返回 n * fact(n - 1)，在函数体中调用函数自身

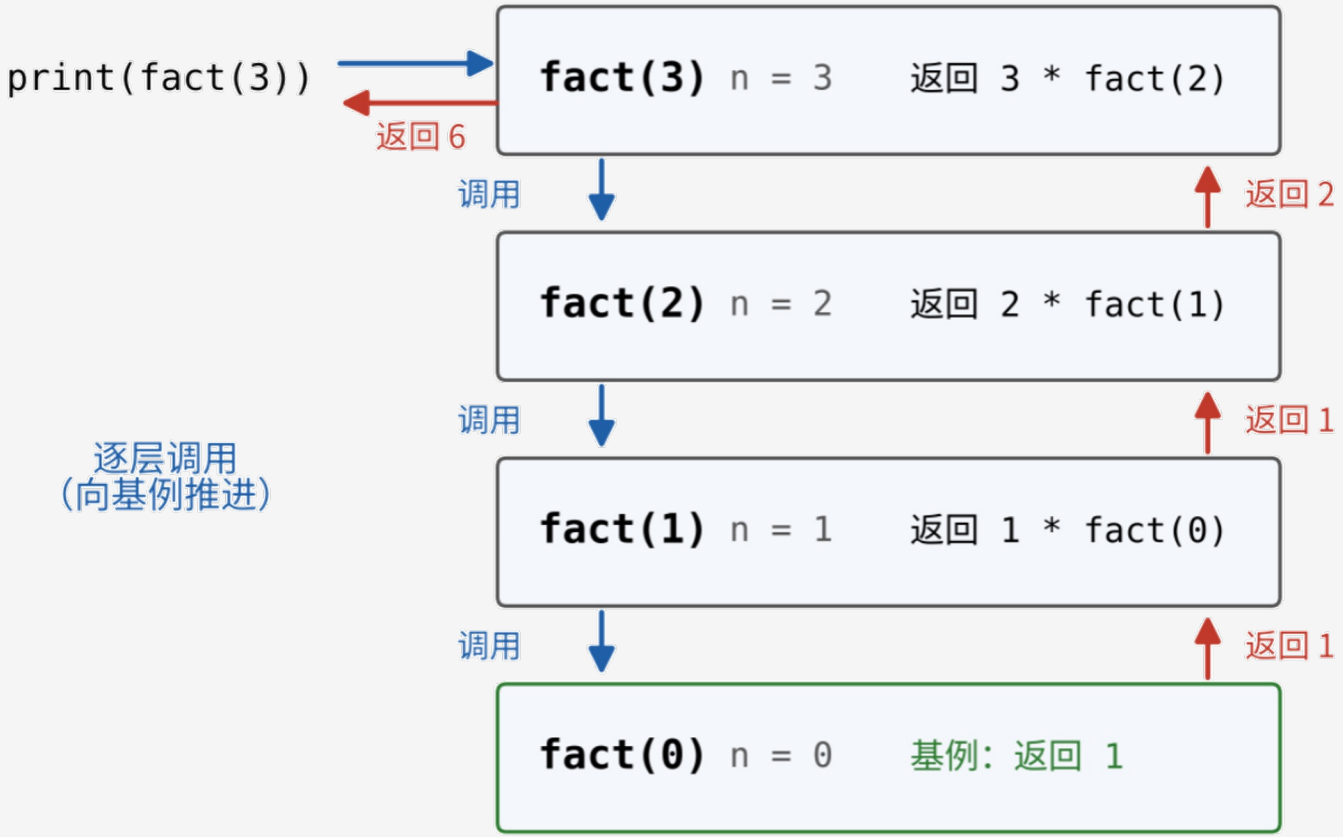
```
def fact(n):
    if n == 0:
        return 1
    else:
        return n * fact(n - 1)
```

函数的递归

- 递归的执行分为两个阶段：
  - 逐层调用：每一次调用都暂停在 n * fact(n - 1) 处，等待 fact(n - 1) 的结果，同时以更小的实参调用自身，直至到达基例
  - 逐层返回：基例直接返回结果，此后各层依次得到下一层的返回值，完成乘法运算，再将结果返回给上一层，直至最外层

函数的递归

- 示例：调用 fact(3) 的过程



- 蓝色箭头：逐层调用，fact(3) → fact(2) → fact(1) → fact(0)
- 绿色框：到达基例 fact(0)，直接返回 1
- 红色箭头：逐层返回，fact(1) 得到 1×1=1，fact(2) 得到 2×1=2，fact(3) 得到 3×2=6

函数的递归

- 设计递归函数的一般步骤
  - 确定递归关系：分析如何将原问题转化为一个或多个规模更小的同类问题，并写出两者之间的关系
  - 确定基例：找出规模最小、可以直接得出结果的情形，作为递归结束的条件
  - 检查递归是否向基例靠近：确认每一次递归调用的实参都比上一次更接近基例，保证递归最终能够结束
- 示例：求正整数各位数字之和

1024 的各位数字之和为 1+0+2+4=7
如何用递归实现？

函数的递归

- 三个步骤

步骤	分析
确定递归关系	1024 的各位数字之和 = 个位数字 4 + 102 的各位数字之和；个位数字为 `n % 10`，去掉个位后的数为 `n // 10`
确定基例	`n` 只有一位时（`n < 10`），各位数字之和即为 `n` 本身
检查是否向基例靠近	每次调用的实参由 `n` 变为 `n // 10`，位数减少一位，最终变为一位数

```
def digit_sum(n):
    if n < 10:
        return n
    else:
        return n % 10 + digit_sum(n // 10)

print(digit_sum(1024))
print(digit_sum(5))
```

函数的递归

- 示例：斐波那契数列
  - 斐波那契数列的定义如下，编写函数求第 n 项 F(n) 的值

$$F(n) = \begin{cases} 1, & n = 1 \text{ 或 } n = 2 \\ F(n-1) + F(n-2), & n \geq 3 \end{cases}$$

步骤	分析
确定递归关系	$F(n) = F(n-1) + F(n-2)$
确定基例	$F(1) = F(2) = 1$，可直接得出；本例有**两个基例**
检查是否向基例靠近	每次调用的参数变为 $n-1$ 和 $n-2$，逐次减小，最终到达 1 或 2

函数的递归

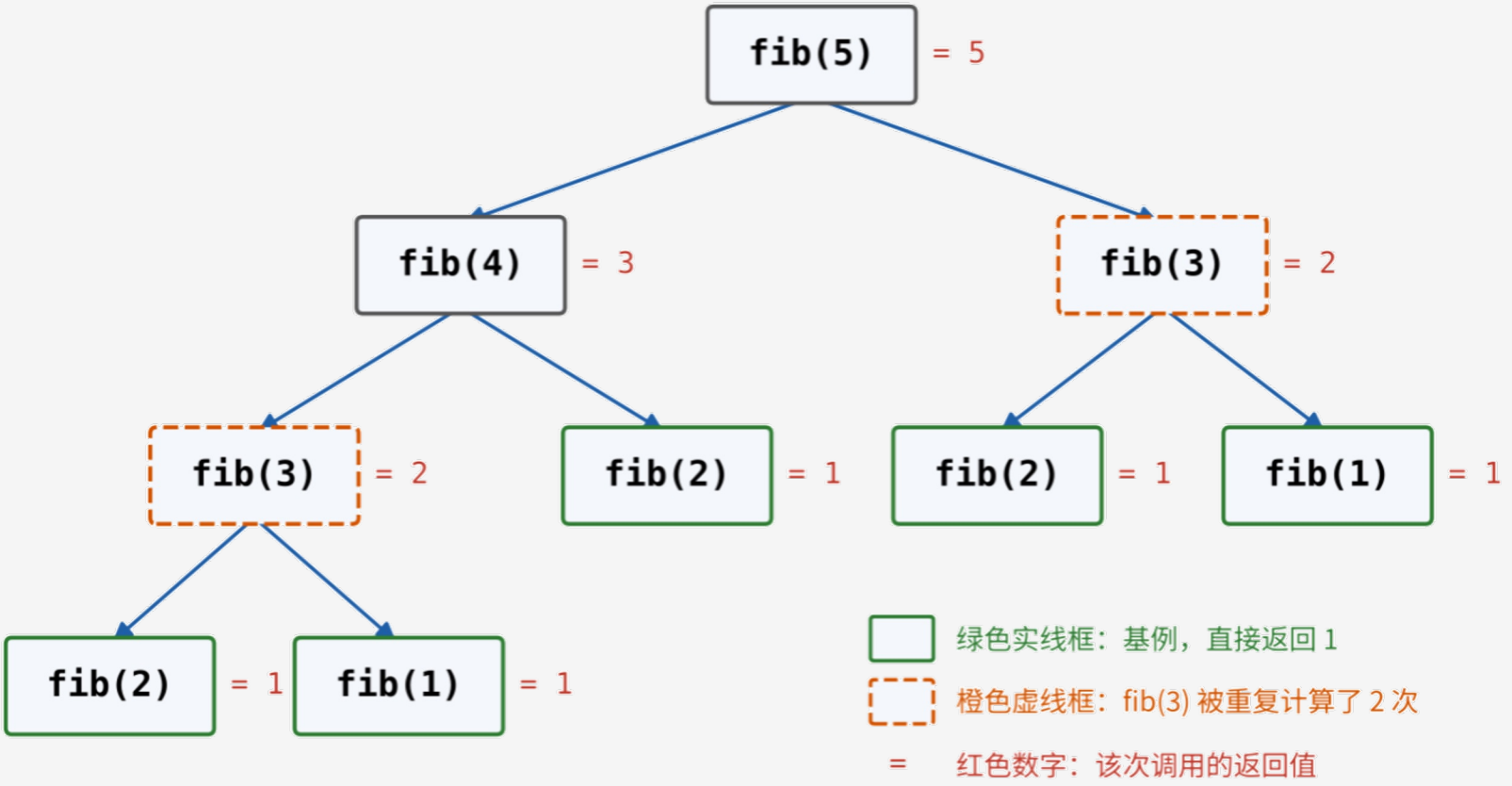
- 示例：斐波那契数列
  - 斐波那契数列的定义如下，编写函数求第 n 项 F(n) 的值

$$F(n) = \begin{cases} 1, & n = 1 \text{ 或 } n = 2 \\ F(n-1) + F(n-2), & n \geq 3 \end{cases}$$

```
def fib(n):
    if n == 1 or n == 2:
        return 1
    else:
        return fib(n - 1) + fib(n - 2)
```

函数的递归

- 示例：斐波那契数列
  - 调用 fib(5) 时的调用关系：



- 每个方框表示一次函数调用，箭头由调用方指向被调用方
- 每一次非基例的调用都会产生两次新的调用：fib(n - 1) 和 fib(n - 2)

函数的递归

- 递归的深度限制
  - 函数每调用一次自身，递归的层数（深度）就增加一层
  - Python 对递归深度设有上限，默认约为 1000 层；超过上限时，程序终止并报告 RecursionError 错误
  - 设置上限的目的是防止函数无限地调用自身，耗尽计算机资源
- 引发 RecursionError 的三种情形：
  - **情形一**：缺少基例
    - 没有结束条件，函数无限地调用自身
  - **情形二**：无法到达基例
    - 斐波那契数列函数，实参由 0 变为 -1、-2……，永远不会等于 1 或 2
  - **情形三**：递归层数过多
    - fib (5000) 需要约 5000 层递归，超过上限

⚠ 对于需要递归很多层的问题，应考虑改用**循环**实现

函数的递归

- 递归的深度限制
  - 函数每调用一次自身，递归的层数（深度）就增加一层
  - Python 对递归深度设有上限，默认约为 1000 层；超过上限时，程序终止并报告 RecursionError 错误
  - 设置上限的目的是防止函数无限地调用自身，耗尽计算机资源
- 引发 RecursionError 的三种情形：
  - **情形一**：缺少基例
    - 没有结束条件，函数无限地调用自身
  - **情形二**：无法到达基例
    - 斐波那契数列函数，实参由 0 变为 -1、-2……，永远不会等于 1 或 2
  - **情形三**：递归层数过多
    - fib (5000) 需要约 5000 层递归，超过上限

⚠ 对于需要递归很多层的问题，应考虑改用循环实现

函数的递归

- 递归与循环的比较

比较项	递归	循环
代码形式	与数学定义直接对应，简洁易读	需要自行设计变量的更新过程
执行效率	每次调用都有额外开销，且可能重复计算	无函数调用开销，效率较高
规模限制	受递归深度上限约束	无深度限制
适用场景	问题本身具有递归定义、递归层数不多	规模较大或对效率有要求的问题

- 示例：用循环求斐波那契数列的第 n 项

```
def fib_loop(n):
    a, b = 1, 1
    for i in range(n - 2):
        a, b = b, a + b
    return b
```

a、b 始终保存相邻的两项，每次循环向后推进一项

1.6 基础篇练习题

基础篇练习题

- 1. 写出下列程序的运行结果：

```
def show():
    print("A")

print("B")
show()
print("C")
```

运行结果：

B
A
C

基础篇练习题

- 2. 写出下列程序的运行结果：

```
def check(score):
    if score >= 60:
        return "及格"


print(check(85))
print(check(45))
```

运行结果：
及格
None

调用 check(45)：45 < 60，条件不成立，跳过 return 语句，函数体执行结束时没有遇到任何 return，因此返回 None

基础篇练习题

- 3. 写出下列程序的运行结果：

```
def min_max(a, b):
    if a < b:
        return a, b
    return b, a
```

用一个变量 r 接收，r 为元组 (3, 8)

用两个变量接收，返回值依次赋给 small 和 big，分别为 5 和 9

```
r = min_max(8, 3)
print(r)
small, big = min_max(5, 9)
print(small, big)
```

运行结果：

(3, 8)

5 9

基础篇练习题

- 4. 写出下列程序的运行结果：

```
def calc(a, b, c):
    return a - b + c


print(calc(1, 2, 3))
print(calc(1, c=2, b=3))
print(calc(c=1, b=2, a=3))
print(calc(1, 2, b=3))
```

位置参数

位置参数+关键字参数

关键字参数

第四次调用：1、2 按位置依次传给 a、b，随后 b=3 又按名称给 b 传值，同一个形参接收了两个实参

运行结果：

2
0
2
TypeError: calc() got multiple values for argument 'b'

基础篇练习题

- 5. 对于第 4 题中的函数 calc，下列调用语句中会报错的是：

A. `calc(1, 2, 3)`
B. `calc(1, c=2, b=3)`
C. `calc(a=1, 2, 3)`
D. `calc(3, b=1, c=2)`

答案：
C

关键字参数 a=1 写在位置参数 2、3 之前，违反“位置参数必须在关键字参数之前”的规则

基础篇练习题

- 6. 下列函数定义中，会产生语法错误的是：

A. `def f(a, b=1):`

B. `def f(a=1, b):`

C. `def f(a=1, b=2):`

D. `def f(a, *b):`

答案：
B

B 中带默认值的 a=1 位于不带默认值的 b 之前，违反“带默认值的形参必须位于不带默认值的形参之后”的规则

D 中 *b 为可变数量参数，位于普通形参 a 之后

基础篇练习题

- 7. 写出下列程序的运行结果

```
def power(x, n=2):
    return x ** n


print(power(3))
print(power(2, 3))
print(power(n=3, x=2))
```

答案：

9
8
8

基础篇练习题

- 8. 写出下列程序的运行结果

```
def count(*items):
    return len(items)

print(count())
print(count(1, 2, 3))
print(count("a", "b"))
```

答案：
0
3
2

*items 将传入的全部实参组装为元组，len(items) 即为实参个数

基础篇练习题

- 9. 写出下列程序的运行结果

```
x = 10

def f1():
    x = 20

def f2():
    global x
    x = 30

f1()
print(x)
f2()
print(x)
```

新建了局部变量 x，全局变量 x 不受影响

使用 global x 声明后赋值，直接修改全局变量 x

答案：

10

30

基础篇练习题

- 10. 写出下列程序的运行结果

```
words = ["pear", "fig", "banana"]
print(sorted(words, key=lambda w: len(w)))
print(max(words, key=lambda w: len(w)))
```

答案：
['fig', 'pear', 'banana’]
Banana

lambda 函数 lambda w: len(w) 返回单词的长度

基础篇练习题

- 11. 写出下列程序的运行结果：

```
def f(n):
    if n == 1:
        return 1
    else:
        return f(n - 1) + n * 2

print(f(3))
```

答案：
11

逐层调用：f(3) → f(2) → f(1)，到达基例

基础篇练习题

- 11. 写出下列程序的运行结果：

```
def f(n):
    if n == 1:
        return 1
    else:
        return f(n - 1) + n * 2

print(f(3))
```

调用	计算过程	返回值
`f(1)`	基例	1
`f(2)`	`f(1) + 2 * 2` = 1 + 4	5
`f(3)`	`f(2) + 3 * 2` = 5 + 6	11

基础篇练习题

- 12. 补全下列程序，用递归求 x 的 n 次幂（n 为非负整数）

$$x^n = x \times x^{n-1}, \text{ 且 } x^0 = 1$$

```
def power(x, n):
    if ______①______:
        return 1
    else:
        return ______②______
```

答案：

n == 0

x * power(x, n - 1)